



Automating code generation for a new ecosystem: establishing baselines with large language model based code generation for ArkTS and HarmonyOS

Mehmet Cem Aytekin¹ · Fatma Gizem Yılmaz¹ · Mustafa Umut Demirezen²

Received: 13 August 2025 / Accepted: 25 January 2026

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2026

Abstract

Automated code generation has advanced significantly with large language models (LLMs), yet their performance in emerging domain-specific languages for practical software development remains largely untested. This gap is particularly critical for major, rapidly growing platforms like Huawei's HarmonyOS. Powering over 900 million devices, the HarmonyOS ecosystem is shifting to a native-only application environment with the HarmonyOS NEXT update. This shift makes development in its native ArkTS UI language essential, driving an urgent need for effective developer tools. Addressing this industry challenge, we conduct the first systematic evaluation of Large Language Models (LLMs) on their ability to generate valid ArkTS code to accelerate the software development lifecycle and assist developers in mastering this new language. To ground our evaluation in practical scenarios, we introduce two curated datasets: a test dataset, *ArkTS-Test*, and a training dataset for fine-tuning, both derived from common UI development tasks in ArkTS. Using these datasets, we evaluate a diverse range of LLMs—from large-scale proprietary models (e.g., Claude, Gemini, DeepSeek-V3, Qwen3 Coder, from 70b to up to 671B total parameters) to smaller open-source models (7B–14B) and report our observations. Next, we propose a methodology called Iterative Compilation Feedback (ICF), which enables LLMs to autonomously correct their own code by leveraging compiler error messages. Our experiments show that ICF boosts the syntactic Pass@1 of large-scale LLMs to as high as 91%. Furthermore, we show that fine-tuning a small-scale LLM (GPT-4o-mini) and combining it with our ICF method yields results comparable to the best-performing large-scale models. Finally, we conclude with a detailed categorization of compilation errors, identifying which types our ICF method resolves most effectively and which persist due to model knowledge limitations.

Extended author information available on the last page of the article

Published online: 27 February 2026

Springer

Keywords ArkTS · HarmonyOS · Large language models (LLMs) · Code generation · Compiler feedback · Low-resource languages

1 Introduction

Recent years have witnessed significant advances in automated code generation driven by large language models (LLMs) such as GPT-4 and specialized coding models like CodeGen (Jiang et al. 2024). These models have shown impressive capabilities on established benchmarks like HumanEval, which focus on mainstream languages and algorithmic problems. However, a critical gap exists between these benchmarks and real-world software development, particularly for domain-specific languages and UI-centric frameworks.

A prime example of this challenge is Huawei's HarmonyOS, a major software ecosystem powering over 900 million active devices. HarmonyOS is designed to run on a wide range of hardware, including smartphones, wearables, and vehicles, enabling them to work together. The platform is undergoing a definitive transition with the introduction of HarmonyOS NEXT. Unlike previous iterations, this version removes the Linux kernel and the Android Open Source Project (AOSP) compatibility layer entirely, rendering existing Android APKs incompatible. This shift forces a complete migration to a native-only ecosystem, making ArkTS the mandatory and exclusive language for UI development. Although ArkTS is syntactically based on TypeScript, it diverges significantly in execution: it enforces a strict static typing system (disabling dynamic features like the `any` keyword to enable Ahead-of-Time compilation) and utilizes a unique declarative paradigm for state management (e.g., struct-based components vs. class-based). Figure 1 illustrates this domain gap, contrasting the familiar functional patterns of standard web development with the strict, struct-based architecture required by ArkTS. This mismatch causes general-purpose LLMs, trained on standard TypeScript web development, frequently hallucinate syntax that is valid in TypeScript but illegal in the strict ArkTS environment. Consequently, since algorithmic business logic in HarmonyOS largely follows standard, strict TypeScript syntax—a domain where LLMs already demonstrate high proficiency—our study specifically isolates the unique ArkTS UI paradigm where model performance degrades most significantly.

The poor performance of LLMs on ArkTS, which we confirmed in our initial tests across various models, stems from a fundamental mismatch between their training and the demands of ArkTS for its UI development. The training data for many state-of-the-art LLMs is heavily weighted towards function-level, algorithmic tasks. ArkTS development, however, requires a different set of skills centered on UI-specific tasks like component layout, state management, and user interaction. This requires mastering ArkTS-specific concepts like its decorator-based state management (e.g., using `@State` and `@Link` to automatically sync UI with data), the strict architectural rules of its component hierarchy, and its unique library of UI modifiers—a programming paradigm fundamentally different from the algorithmic tasks that dominate most LLM training data.

Standard TypeScript (Web)	HarmonyOS ArkTS (Native)
<pre>// 1. Functional Component const Counter = () => { // React Hook const [c, setC] = useState(0); return (// HTML-like JSX Tags <div style={{pad: 10}}> <button onClick={() => setC(val => val + 1)}> Count: {c} </button> </div>); };</pre>	<pre>// 1. Struct-based Component @Component struct Counter { // Decorator (No Hooks) @State c: number = 0; build() { // Chainable UI Methods Column() { Button('Count: \${this.c}') .onClick(() => { this.c += 1; }) .padding(10) } } }</pre>

Fig. 1 The Domain Gap: A side-by-side comparison. **Left:** Standard TypeScript (React) uses hooks and JSX. **Right:** ArkTS uses structs, decorators, and method chaining. General LLMs frequently hallucinate the left-hand syntax for ArkTS tasks. (Source: Adapted from HarmonyOS Developer Documentation Huawei Developers [2024](#))

While recent work has explored strategies to improve code generation reliability (Hu et al. [2024](#); Li and Yuan [2024](#)), these methods have not been adapted for the specific challenges of a low-resource, UI-centric language like ArkTS. Motivated by this clear industry need, this paper presents, to the best of our knowledge, the first methodological case study on adapting LLMs for ArkTS code generation. We address the question of how to effectively use general-purpose models for a specialized framework that demands domain-specific knowledge and precise syntax. Our contributions are as follows:

- We construct and release a curated dataset of ArkTS instruction–code pairs designed to cover a wide range of ArkTS UI development tasks and ArkUI components.
- We conduct a systematic evaluation of diverse large-scale proprietary and small-scale open models on ArkTS UI code generation using an automated compilation-based syntactic validation pipeline.
- We propose an Iterative Compilation Feedback (ICF) method that uses ArkTS compiler error messages to iteratively refine model outputs until they compile or a maximum number of attempts is reached.
- We perform supervised fine-tuning of a small-scale model on ArkTS instruction–code pairs and evaluate the impact of fine-tuning alone and in combination with ICF.
- We analyze compilation errors produced during ICF, providing a categorization of common failure modes and documenting which error types are more likely to be resolved through iterative compilation feedback.
- For syntactically correct outputs from the best-performing models, we assess semantic correctness through manual runtime testing on a HarmonyOS virtual device.

The remainder of this paper is organized as follows. Section 2 reviews related work in code generation, debugging-guided approaches, and adaptation to domain-specific languages. Section 3 describes our methodology, including the construction of the curated ArkTS dataset, the evaluation setup, and the Iterative Compilation Feedback (ICF) process. Section 4 reports our experimental results, covering baseline performance, ICF improvements, fine-tuning outcomes, and detailed error analysis, followed by semantic validation of the best-performing models. Section 5 interprets the results, outlines limitations, and identifies directions for future work. Finally, Section 6 summarizes our contributions and key findings.

2 Related work

Research on LLM-assisted code generation has expanded rapidly, establishing new paradigms for automated software development. While general-purpose models have demonstrated proficiency in mainstream programming tasks, applying them to specialized ecosystems involves distinct challenges. In this section, we review the existing literature through four primary lenses relevant to our study: standard benchmarks, structural UI generation strategies, adaptation for low-resource languages, and feedback-driven refinement.

2.1 Benchmarks and general code generation

Standard benchmarks such as HumanEval (Hendrycks et al. 2021), APPS (Austin et al. 2021), and MBPP (Austin et al. 2021) provide controlled settings for evaluating functional correctness and syntactic accuracy. These typically pose algorithmic challenges and verify generated solutions against hidden test cases. Other datasets like CodeXGLUE (Lu et al. 2021), CoNaLa (Yin and Neubig 2018), SPoC (Kulal et al. 2019), and CodeSearchNet (Husain et al. 2019) target specific skills such as code completion, semantic translation, and retrieval. Evaluation in these settings is often based on Pass@k, compilation success rate, and runtime correctness.

While these resources have advanced general-purpose code generation, they remain largely centered on algorithmic or data structure-driven problems. This makes them ill-suited for languages like ArkTS, where successful development depends on component composition, event-driven state management, and platform-specific API usage—capabilities absent from most benchmarks. Even large-scale datasets like OpenCodeInstruct (Ahmad et al. 2025), which offer millions of instruction–code pairs, remain predominantly algorithmic (e.g., scheduling tasks). This limits their direct applicability to the multi-component, interface-bound tasks that characterize ArkTS development.

2.2 Structural and UI-aware generation

To address the complexity of UI development, structured generation methods introduce explicit reasoning or planning phases before code emission (Jiang et al. 2023b). In such approaches, the model first produces a high-level plan—for example, out-

lining UI component hierarchies—before generating the implementation. This decomposition has yielded notable Pass@1 gains in methods like Planning-Guided Transformer Decoding (Zhang et al. 2023).

These strategies are particularly relevant to ArkTS, where respecting parent–child component nesting is critical. Hierarchy-aware frameworks like DesignCoder (Chen et al. 2025) and UICopilot (Gui et al. 2025) employ multi-stage generation pipelines to improve structural coherence for complex interfaces. Furthermore, LayoutCoder (Wu et al. 2025) emphasizes visual-structural alignment by explicitly parsing UI layout trees, while WAFFLE (Liang et al. 2024) integrates structure-aware attention to match structural patterns with visual designs. While these methods improve structure, they do not inherently solve the issue of domain-specific syntax hallucinations in a new ecosystem.

2.3 Code generation for low-resource domain-specific languages

A critical challenge in applying LLMs to ArkTS is the “domain gap.” Prior works such as Arakelyan et al. (2023) and Nashid et al. (2023) highlight that models experience notable performance drops when applied to previously unseen language distributions. While models excel in high-resource languages like Python or Java, performance degrades in Domain-Specific Languages (DSLs) with sparse public repositories.

ArkTS presents a unique distributional challenge: it utilizes a “high-resource syntax” (TypeScript) applied to a “low-resource framework” (HarmonyOS). This causes general-purpose models to conflate the two, producing code that is valid TypeScript but illegal in the strict ArkTS environment (e.g., prohibited dynamic typing). While retrieval-based few-shot adaptation can offer modest gains (Nashid et al. 2023), it falters when syntax and usage patterns diverge sharply from training data, underscoring the need for targeted adaptation strategies like the one presented in this study.

2.4 Compiler and execution feedback loops

Even with careful planning, generated code often fails at runtime, motivating feedback-driven refinement. Debugging-guided methods incorporate execution results into the generation loop (Hu et al. 2024), prompting models to insert diagnostic statements for error localization. Systems like TestChain (Li and Yuan 2024) use collaborating agents to iteratively refine implementations based on test feedback. Empirical findings confirm that rich feedback—such as stack traces or compiler messages—substantially improves reliability (Yeo et al. 2024; Chen et al. 2022).

However, existing feedback approaches typically focus on correcting *logical* errors in languages where the model already masters the syntax (e.g., fixing a Python sorting algorithm). Our work adapts this methodology to a new context: correcting *hallucinated APIs* and *strict type violations* in a DSL. Unlike standard debugging, our Iterative Compilation Feedback (ICF) method leverages compiler diagnostics not just to fix bugs, but as a knowledge grounding mechanism, teaching the model the boundaries of the ArkTS framework that were absent in its pre-training data.

3 Methodology

This section outlines our methodology for evaluating Large Language Models on ArkTS code generation tasks. We begin in Section 3.1 by introducing the *ArkTS-Test* dataset, detailing its design goals, curation process, and final composition. Section 3.2 describes the selection of LLMs across different architectures and parameter scales to enable broad comparative analysis. Section 3.3 defines our primary evaluation metric—Pass@1—and explains its relationship to pass@k. Section 3.4 presents our two-tier correctness assessment framework, combining automated syntactic validation with manual semantic validation. Section 3.5 then introduces the Iterative Compilation Feedback (ICF) method, which iteratively improves code by leveraging compiler error feedback, while Section 3.6 details how we evaluate ICF’s effectiveness using the Success@k metric. Finally, Section 3.7 reports our supervised fine-tuning experiment on a small-scale model to assess the gains from targeted ArkTS-specific training.

3.1 Evaluation dataset: ArkTS-Test

ArkTS employs a declarative UI paradigm, where the user interface is defined as a function of the application’s state. The fundamental building blocks are components (e.g., `Button`, `Text`, `Image`) organized hierarchically within layout containers. To ensure *ArkTS-Test* reflects practical development scenarios, we established a rigorous data curation process.

3.1.1 Inclusion criteria

To ensure the dataset represents practical HarmonyOS ArkTS UI development tasks, we derived task patterns from the official OpenHarmony Gitee repository (OpenHarmony 2024) and established the following strict inclusion criteria:

- **Decorator-Driven State Management:** Every task must require at least one ArkTS-specific state decorator (e.g., `@State`, `@Link`, or `@Prop`) to verify the model’s understanding of declarative state-driven UI updates.
- **Architectural Nesting:** Tasks must involve nested layout containers (e.g., a `List` inside a `Column`) to test the model’s ability to produce valid hierarchical UI structure.
- **Component Coverage:** Across the dataset, tasks must collectively cover a broad range of ArkUI components and layouts (more than 20 distinct ArkUI primitives), spanning text/display components, input controls, list-based views, navigation patterns, and common interactive elements.
- **Compilation Validity:** All reference implementations must compile with zero errors in DevEco Studio 4.0+ (the official IDE), ensuring adherence to ArkTS typing and framework rules.

We excluded tasks involving only algorithmic challenges, data processing, or business logic without UI component interaction.

3.1.2 Dataset composition

The evaluation set consists of 100 natural language instructions that describe ArkTS UI tasks. For each instruction, we created a corresponding reference implementation that is used as the ground truth during validation in our experiments. The public release includes the instruction list, and the repository also provides a separate collection of instruction–code examples used for model training.

The dataset does not include automated runnable unit or UI test cases for each data point. This is because many tasks include visual layout and interaction requirements that are not reliably captured by standard unit-test assertions.

Data quality was ensured through the following measures: (1) strict compilation validity testing (all reference implementations compile with zero errors in DevEco Studio), (2) manual verification by expert ArkTS developers against the instruction requirements, and (3) adherence to official ArkUI primitives and documented usage patterns to avoid non-canonical or ambiguous behavior.

3.2 Model selection

For our evaluation, we selected a diverse range of Large Language Models (LLMs) to capture a comprehensive snapshot of the current AI landscape and its applicability to ArkTS code generation. A detailed list of the models, their development families, and their specific features is presented in Table 1. The rationale for this diverse

Table 1 Large language model variants and configurations evaluated for ArkTS code generation

Family	Model variant	Specific features
Claude	Claude-3.7-Sonnet	High-performance model balancing speed and intelligence, released in February 2025 (Anthropic 2025a)
	Claude Sonnet 4	Next-generation model with state-of-the-art reasoning and code generation capabilities (Anthropic 2025b)
DeepSeek	DeepSeek-R1-Distill-Llama-8b	Compact 8B parameter model distilled from DeepSeek-R1 for efficient reasoning (DeepSeekAI 2025)
	DeepSeek-V3-0324	Mixture-of-experts architecture with 671B parameters for complex reasoning (DeepSeek AI 2025)
Google	Gemini 2.5 Pro	Next-generation multimodal model designed for advanced reasoning across text, images, and code (Google DeepMind 2025)
	Gemma3-12b	Lightweight open model developed by Google, optimized for high-speed inference (Google 2025)
GPT	GPT-4o-mini	Compact variant optimized for lower resource utilization (OpenAI 2025)
Qwen	Qwen3 Coder	Mixture-of-Experts (MoE) model (480B total) for long-context reasoning (Qwen Team 2025b)
	Qwen2.5-Coder-14B-Instruct	Smaller coding-focused version with efficient processing (Qwen Team 2025a)
Mistral	Mistral-7b Instruct	Streamlined for lightweight applications, balancing speed and accuracy (Jiang et al. 2023a)

selection is to assess performance across different model architectures, sizes, and specializations, from state-of-the-art proprietary systems to accessible open-source alternatives.

To systematically analyze the impact of model scale on performance, we classify the selected models into two main categories, as detailed in Table 2: Large-Scale and Small-Scale Models. The first category, Large-Scale Models, includes flagship proprietary models from major AI labs such as Google's Gemini 2.5 Pro and Anthropic's Claude series (Claude-3.7-Sonnet and Claude Sonnet 4). This group also contains powerful Mixture-of-Experts (MoE) models like Deepseek-chat-v3-0324 and Qwen3 Coder. With parameter counts estimated to be in the hundreds of billions, these models represent the current upper bound of reasoning and code generation capabilities and serve as a benchmark for what is maximally achievable.

The second category, Small-Scale Models, consists of smaller, predominantly open-source models with parameter counts typically ranging from 7B to 16B. This group includes popular and efficient models such as Mistral-7b Instruct, Google's Gemma3-12b, and various code-centric variants from the DeepSeek and Qwen families. These models are critically important to evaluate as their smaller footprint makes them more practical candidates for fine-tuning, local deployment, and integration into developer tools where resource efficiency is a key concern. By evaluating both categories, our study aims to create a holistic picture, comparing the raw power of large-scale models against the practical efficiency and fine-tuning potential of their smaller counterparts in the specialized domain of ArkTS development.

3.3 Pass@1 as the evaluation metric

To evaluate the code generation performance of the LLMs, we utilize **Pass@1** as our primary evaluation metric. The standard metric for code generation assessments is $\text{pass}@k$ (Kulal et al. 2019), which generates n code solutions per problem and considers a problem solved if at least k of those solutions are correct. The formal definition is:

Table 2 Classification of evaluated LLMs by parameter size

Category	Model variant	Parameters
Large-Scale Models	Deepseek-V3-0324	671B
	Qwen3 Coder	480B
	Claude Sonnet 4	> 350B*
	Gemini 2.5 Pro	> 250B*
	Claude-3.7-Sonnet	~ 175B*
Small-Scale Models	Qwen2.5-Coder-14B-Instruct	14B
	Gemma3-12b	12B
	DeepSeek-R1-Distill-Llama-8b	8B
	GPT-4o-mini	8B*
	Mistral-7b Instruct	7B

*Estimated parameter count; not officially disclosed

$$\text{Pass@}k = \mathbb{E} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right] \quad (1)$$

In this equation:

- n is the total number of generated solutions per problem.
- c is the count of solutions that pass the tests.
- k is the minimum number of correct solutions required.

In our study, we employ a strict instance of this metric where $n = 1$ and $k = 1$, meaning we generate only a single code solution for each instruction and evaluate whether that one solution is correct. A high Pass@1 score indicates a high degree of “first-try” reliability, which is crucial for practical developer assistance tools where users expect an immediate, correct answer without regenerating multiple times.

3.4 Correctness assessment framework

To evaluate the performance of the models, we assess correctness at two distinct levels: syntactic and semantic. This framework allows us to first establish a baseline of each model’s ability to generate valid ArkTS syntax. Following this, we perform a deeper, manual analysis of semantic correctness on the outputs from only the best-performing models to gauge their practical utility. This overall process is illustrated in Fig. 2.

3.4.1 Automated syntactic validation

Syntactic accuracy measures whether the generated code is valid according to the ArkTS language specification. A solution is deemed syntactically correct if it compiles with zero errors. This metric is calculated for all models using a fully automated pipeline orchestrated by a custom Python script.

1. **Environment Emulation:** The script first configures the necessary environment variables (NODE_HOME, JAVA_HOME, etc.) to precisely mirror the local environment of DevEco Studio, Huawei’s official Integrated Development Environment (IDE) for HarmonyOS development. This ensures the compilation is authentic.
2. **Safe Code Injection:** The LLM-generated code is then written into a pre-configured template project. We employ a file caching system that saves the original content of the target .ets file before overwriting it, ensuring that the project can be reliably restored after each compilation attempt.
3. **Compiler Invocation:** Using a subprocess, the script programmatically executes the HarmonyOS build tool, hvmgrw, to compile the entire project.
4. **Result Analysis:** The script captures the exit code of the compilation. An exit code of 0 signifies a successful build, and the attempt is marked as syntactically

Instruction

Design a arkTS app that includes a login page with email and password text fields. Upon pressing the login button, validate the input and display a success message if the login is successful.

translated
into arkTS
Code

ArkTS Code

```
@Entry
@Component
struct LoginScreen {
  @State email: string = ''
  @State password: string = ''
  @State message: string = ''
  @State isSuccess: boolean = false

  build() {
    Column() {
      // Email Input
      Text('Email:')
        .fontSize(18)
        .fontWeight(FontWeight.Bold)
        .margin({ bottom: 10 })

      TextInput({ placeholder: 'Enter your email' })
        .width('80%')
        .margin({ bottom: 20 })
        .onChange(value: string) => {
          this.email = value
        })

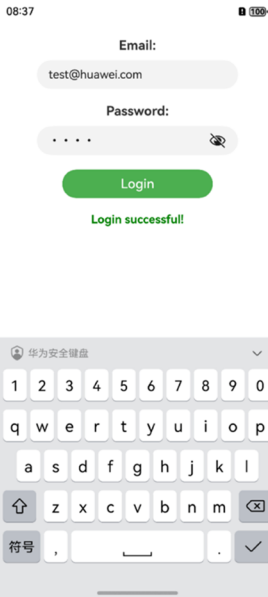
      // Password Input
      Text('Password:')
        .fontSize(18)
        .margin({ bottom: 10 })

      PasswordInput()
        .margin({ bottom: 10 })

      LoginButton()
        .margin({ bottom: 10 })

      MessageText()
        .margin({ bottom: 10 })
    }
  }
}
```

Result in Virtual Device



run on Virtual
Harmony OS
Mobile Phone

Fig. 2 Example of a single data point from the *ArkTS-Test*, illustrating the end-to-end process: a natural language instruction is translated into ArkTS code, which is then compiled and run on a HarmonyOS virtual device for semantic validation

correct. Any non-zero exit code indicates failure. The final syntactic accuracy score for each model is the percentage of prompts for which it generated compilable code.

3.4.2 Manual semantic validation

To assess the practical utility of the most capable models, we performed a manual semantic evaluation on their syntactically correct outputs. This intensive process measures whether a compiled application functions exactly as described in the natural language prompt.

For each syntactically correct solution generated by our top-performing models, a designated human tester deployed the application to a HarmonyOS virtual device. The tester performed a comprehensive functional review, carefully comparing the application's behavior and appearance against every requirement detailed in the prompt. This included verifying UI layout, event handling, and state changes.

An output was considered semantically correct only if it perfectly implemented all specified requirements. The semantic accuracy score for these selected models is calculated as the percentage of their syntactically correct solutions that also pass this manual verification.

3.5 The iterative compilation feedback (ICF) method

To address the challenge of generating syntactically correct code for a low-resource language like ArkTS, we designed a technique called Iterative Compilation Feedback (ICF). The purpose of this method is to enable an LLM to autonomously refine its own code by using the output from the ArkTS compiler as a guide.

The ICF method operates as a stateless, iterative loop that systematically attempts to resolve compilation errors. The complete workflow is visualized in Fig. 3. The process begins with an initial code generation based on the user instruction. This code is then passed to our automated compiler script. If the compilation fails, a new, self-contained prompt is constructed for the next iteration. This new prompt includes the original instruction, the full erroneous code from the failed attempt, and the complete error message returned by the compiler. This loop continues, feeding increasingly specific debugging information back to the model, until either a compilable solution is generated or a maximum limit of k iterations is reached.

The effectiveness of this loop relies on providing clear, structured context in each corrective iteration. Since each call to the model is an independent interaction, the prompt must contain all necessary information for the model to perform the debugging task. Following a failed compilation, the prompt for the next iteration is assembled using the following template:

Your previous attempt to generate ArkTS code failed to compile. Analyze the provided code and the compiler error message to generate a fully corrected version that meets the original instruction.

Original Instruction:

[Original natural language instruction from ArkTS-Test]

Failing Code:

`""arkts`

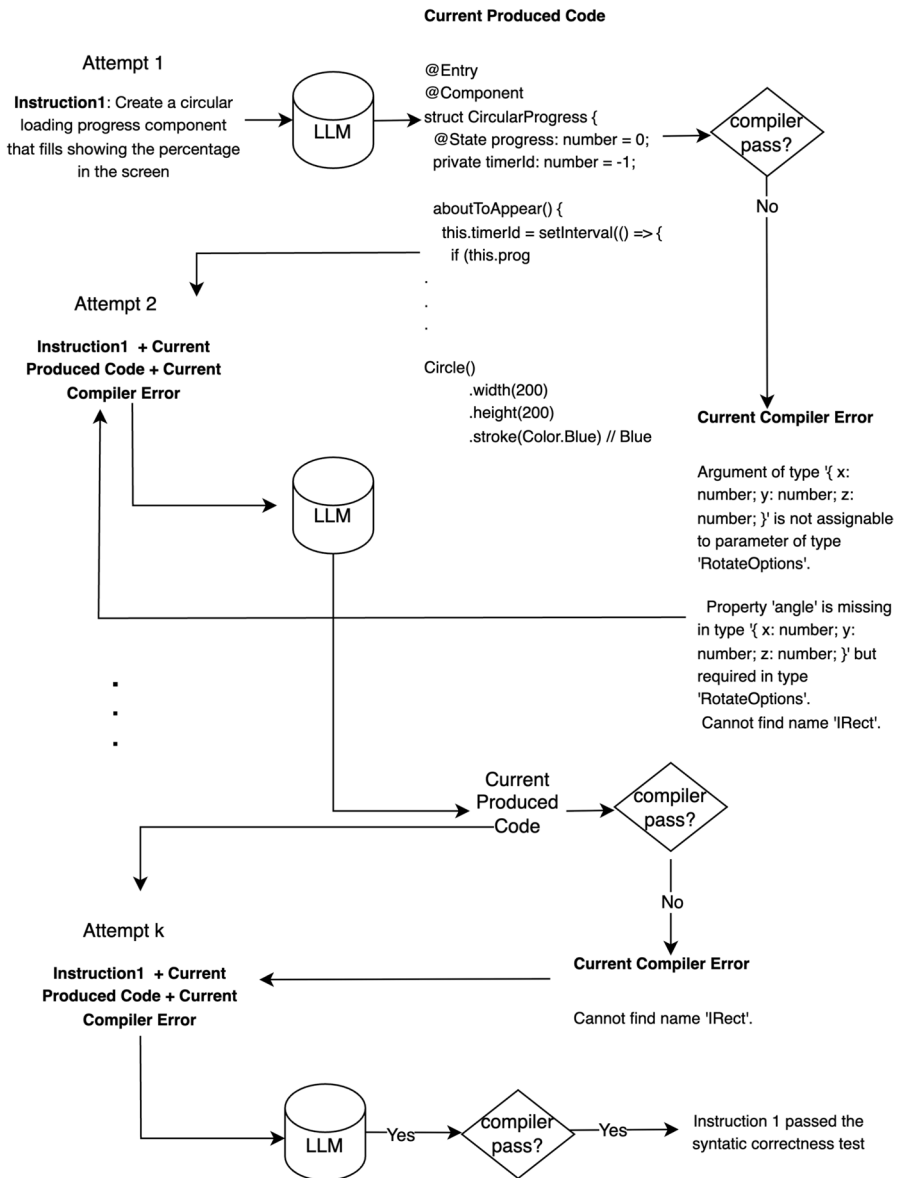


Fig. 3 The workflow of the Iterative Compilation Feedback (ICF) method. An LLM's code output is compiled. If it fails, the code and the resulting compiler error are fed back into the model to generate a corrected version in the next attempt. This loop continues until compilation succeeds or a maximum number of attempts is reached

[The full, erroneous ArkTS code from the previous attempt]

""

Compiler Error:

""

[The raw text output of the compiler error message]

""

Provide the fully corrected ArkTS code:

3.6 Evaluating ICF methodology

While our primary **Pass@1** metric (defined in Section 3.3) effectively measures the first-try reliability of base models, it is insufficient for evaluating the iterative nature of our ICF method. The goal of ICF is to measure a model's ability to achieve correctness through a sequence of self-corrections.

To capture this, we introduce a metric to evaluate the iterative process: the **Cumulative Success Rate at k Attempts**, which we denote as *Success@ k* .

- *Success@ k* : The percentage of instructions for which the ICF method generates a syntactically correct solution in k or fewer attempts.
- k : Represents the attempt number in the iterative feedback loop, up to a predefined maximum of n attempts in our experiments.

It is crucial to distinguish *Success@ k* from the standard *pass@ k* metric. The standard *pass@ k* evaluates success across multiple *independent* code samples generated from scratch. In contrast, our *Success@ k* tracks the progress of a single, *evolving* solution as it is sequentially refined by compiler feedback. This metric is therefore a direct measure of the debugging and refinement capability that ICF provides.

To ensure statistical robustness, we employ a bootstrap analysis on the numerical results of our experiment. The procedure is as follows:

1. The ICF experiment is performed on each instruction in *ArkTS-Test* to produce a master list of outcomes. This list contains only the integer number of attempts required for each instruction to compile successfully (or infinity if it never succeeded).

4. From this master list of numerical outcomes, 2,000 bootstrap samples are generated. Each sample is a new list of numbers created by sampling from the master list with replacement.
3. The *Success@k* curve (for $k = 1$ to n) is calculated for each of the 2,000 bootstrap samples, yielding a distribution of performance curves.
4. The final Mean *Success@k* Curve is the average of this distribution. The 95% Confidence Interval is determined from the 2.5th and 97.5th percentiles of the distribution for each value of k .

3.7 Fine-tuning a small-scale model

To evaluate the potential of a smaller, more resource-efficient model to learn the nuances of ArkTS, we conducted a supervised fine-tuning experiment on GPT-4o-mini. For this process, we utilized OpenAI's dedicated fine-tuning service. While we configured the training with various hyperparameter settings, the detailed analysis of these configurations is presented in the Experiments section. *ArkTS-Test* (described in Section 3.1) was strictly held out and used exclusively as the test set to evaluate the performance of the fine-tuned model.

A key challenge was the creation of a high-quality training dataset. We sourced our data primarily from the official OpenHarmony documentation repository on Gitee (OpenHarmony 2024), China's biggest open source codebase equivalent to GitHub. This repository contains thousands of developer-verified code examples for various ArkTS UI components, often accompanied by visual aids like images or videos that demonstrate the resulting behavior. We focused on examples that utilized UI components similar to those in our test data to ensure relevance. Since the repository provides formal explanations rather than user-centric prompts, we manually transformed those explanations into suitable instructions. For example, a formal explanation like:

"The top navigation bar or side navigation bar can be set to be scrollable if the screen width cannot fully accommodate all the tabs. With a scrollable navigation bar, users can reveal tabs beyond the visible area by touching or swiping on the navigation bar."

was converted into a direct user instruction, such as:

"Create a set of tabs in arkTS where the navigation bar is scrollable. If there are too many tabs to fit on the screen, I want to be able to swipe horizontally to see the rest of the tabs."

We then paired this newly created instruction with its corresponding verified code from the repository. The series of images or short videos accompanying each example in the repository were used to verify that our generated instruction accurately reflected the code's semantic function. Through a careful manual analysis of the repository, we constructed a training set of 1000 high-quality instruction-code pairs. We verified that no instruction or code in our training set overlapped with the con-

tent of our test set. The supervised fine-tuning process was executed by formatting our dataset into the required instruction-response structure and using OpenAI's API. During training, the model learns to map the natural language instructions to their corresponding ArkTS code implementations. The training objective minimizes the cross-entropy loss between the model's predicted code tokens and the actual target code tokens, while the instruction tokens are masked from the loss computation. This process is designed to specifically teach the model the patterns, syntax, and semantics of generating ArkTS code from developer-like queries.

4 Experimental results and discussion

In this section, we present our experimental results. We begin with the performance evaluation of base LLMs in Section 4.1, establishing baseline capabilities across our diverse model selection. Following this, Section 4.2 presents our novel Iterative Compilation Feedback (ICF) methodology results, demonstrating how compiler error messages enable autonomous code correction and substantially improve syntactic reliability for both large-scale and small-scale models. Section 4.3 details the fine-tuning experiments on GPT-4o-mini with different hyperparameter configurations and its enhancement through ICF methodology. We then examine the syntactic error patterns encountered by the best-performing models during the ICF process in Section 4.4, providing insights into which error categories are effectively resolved and which remain challenging for current LLM-based approaches. We conclude our analysis in Section 4.5 by conducting semantic validation of our top-performing models, evaluating whether syntactically correct code fulfills the complete functional requirements through manual testing on HarmonyOS virtual devices.

4.1 Performance of base LLMs

Our initial evaluation focused on establishing a performance baseline for the selected LLMs on *ArkTS-Test*. In this phase, we measured the **Pass@1** of each model, which, as defined in Section 3.3, assesses the model's ability to generate a syntactically correct solution on its first attempt. The results, detailed in Table 3, confirm our hypothesis that generating valid ArkTS code presents a significant challenge for models

Table 3 Pass@1 scores for different models

Model	Pass@1
Qwen2.5-Coder-14B-Instruct	0.01
Qwen3 Coder	0.09
Mistral-7b Instruct	0.00
Gemma3-12b	0.01
Claude-3.7-Sonnet	0.27
Claude Sonnet 4	0.35
Gemini 2.5 Pro	0.11
GPT-4o-mini	0.04
Deepseek-V3-0324	0.15
DeepSeek-R1-Distill-Llama-8B	0.00

not specifically trained on this ecosystem. The overall performance was strikingly low, with no model surpassing a 35% Pass@1 rate. This finding underscores the severe limitations of leveraging general-purpose LLMs for a low-resource, domain-specific language. The declarative UI paradigm, specific component libraries, and unique state management syntax of ArkTS represent a significant departure from the more common algorithmic and web-centric tasks that dominate the training data of most LLMs. A clear performance hierarchy emerged, correlating strongly with model scale. The Large-Scale Models demonstrated superior, yet still limited, capabilities. Claude Sonnet 4 led the group with a Pass@1 of 35%, followed by Claude-3.7-Sonnet at 27%, Deepseek-V3-0324 at 15%, and Gemini 2.5 Pro at 11%. Their advanced general reasoning abilities appear to provide some advantage in deciphering the requirements for this unfamiliar language, yet their success rate remains too low for reliable, practical use. In contrast, the Small-Scale Models, including those specifically optimized for code, performed exceptionally poorly. Models such as Mistral-7b Instruct (0%), Gemma3-12b (1%), and GPT-4o-mini (4%) were almost entirely unable to produce compilable code. Perhaps more revealing is the poor performance of a specialized recent coding model like Qwen3 Coder (9%). This suggests that their optimization for mainstream programming languages and problem-solving paradigms does not translate to the distinct challenges of domain-specific UI development in ArkTS. These baseline results demonstrate that single-attempt code generation is not a viable strategy for applying LLMs to the HarmonyOS ecosystem. The systemic failure across a diverse range of models validates the need for more sophisticated approaches. This critical performance gap motivates our investigation into the methodologies presented in the following sections: the Iterative Compilation Feedback (ICF) technique and targeted fine-tuning.

4.2 ICF method evaluation: achieving syntactic reliability

Our Iterative Compilation Feedback (ICF) method demonstrates remarkable effectiveness in improving code generation reliability across all evaluated models. This section presents the results of applying ICF to each base model, showing how compiler feedback enables autonomous error correction and substantially improves syntactic correctness.

4.2.1 Large-scale model performance

The results demonstrate that ICF transforms syntactic code generation reliability across all large-scale models. As defined in Section 3.4.1, these Success@k measurements evaluate the percentage of instructions for which models generate syntactically correct ArkTS code that compiles with zero errors. **Claude Sonnet 4** achieves the most impressive absolute performance, improving from 35% base syntactic Pass@1 to 91% after 10 iterations—a remarkable +56% absolute improvement and $2.6\times$ relative gain that makes it a candidate for practical use. **Deepseek-V3-0324** shows the most dramatic relative improvement, jumping from just 15% to 83% (+68% absolute, $5.5\times$ relative), while **Claude-3.7-Sonnet** achieves a substantial +53% improvement ($3.0\times$ gain) and **Gemini 2.5 Pro** demonstrates a +60% improvement

($6.5\times$ gain). Notably, **Qwen3 Coder** exhibits the highest relative improvement at $7.3\times$, though starting from the lowest base of 9%.

A striking pattern emerges in the convergence behavior visible in Fig. 4, where all models achieve their most substantial gains within the first 2-3 iterations. Claude Sonnet 4 jumps from 35% to 82% by iteration 3, capturing 84% of its total improvement early. This suggests that many ArkTS compilation errors are relatively straightforward to resolve once the model receives specific compiler feedback, validating our hypothesis that domain-specific compiler error messages can effectively guide autonomous syntactic debugging. Claude Sonnet 4 demonstrates not only the highest absolute performance but also the most consistent behavior, evidenced by its relatively tight confidence intervals throughout the curve. Its performance plateaus around iteration 5-6 at $\sim 89\%$, suggesting it efficiently resolves most correctable errors quickly. Deepseek-V3-0324 exhibits a unique pattern of sustained improvement throughout all 10 iterations, showing continued gains even when other models plateau. This model demonstrates superior error interpretation capabilities that allow it to tackle increasingly complex compilation issues. Despite starting with the lowest base Pass@1 among large models (15%), it ultimately achieves the second-highest final performance (83%), representing the largest absolute improvement ($+68\%$) in our evaluation.

Qwen3 Coder presents an intriguing case study in the limitations of domain-specific optimization. Despite being explicitly designed for code generation, it underperforms compared to general-purpose models like Claude variants. Starting at just 9% base Pass@1, it reaches 66%—substantial improvement ($7.3\times$) but still trailing

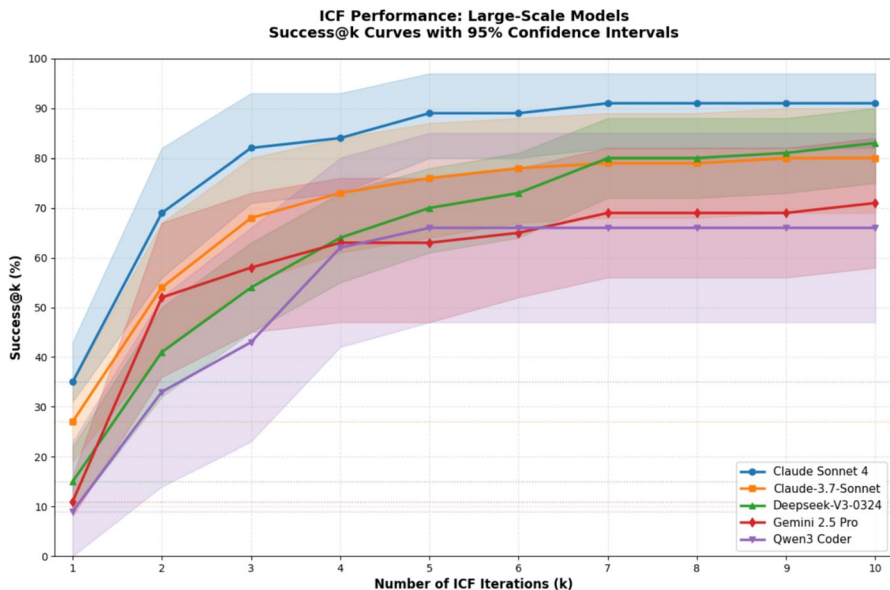


Fig. 4 ICF Success@k curves for large-scale models showing cumulative syntactic accuracy (compilation success) across iterations. The shaded regions represent 95% confidence intervals from bootstrap analysis with 2,000 iterations. All models demonstrate substantial improvement in generating compilable ArkTS code, with Claude Sonnet 4 achieving the highest final syntactic accuracy at 91%

general models. This suggests that specialized training on mainstream programming languages may not transfer effectively to domain-specific UI frameworks like ArkTS. The confidence intervals reveal important insights about model reliability. Claude models show consistently narrow confidence bands, with Claude Sonnet 4 maintaining intervals of approximately $\pm 6 - 8\%$ throughout most iterations, indicating predictable and stable performance across different prompt variations. In contrast, Qwen3 Coder and Gemini 2.5 Pro exhibit wider confidence intervals, particularly in early iterations (e.g., Qwen3 showing $\pm 11 - 19\%$ variance), suggesting more variable performance that could impact practical deployment reliability. The performance curves suggest an optimal stopping point around 5-7 iterations for most models, where the rate of improvement significantly diminishes. For practical implementation, 3 iterations would capture $\sim 75 - 85\%$ of the total improvement for top models, 5 iterations achieves near-maximum performance while maintaining reasonable computational cost, and beyond 7 iterations shows minimal additional gains ($< 3\%$) across all models. Perhaps the most counterintuitive finding is that general-purpose reasoning models significantly outperform coding-specialized models in this domain-specific context. The performance ranking challenges conventional wisdom, with Claude Sonnet 4 leading, followed by Deepseek-V3-0324, Claude-3.7-Sonnet, Gemini 2.5 Pro, and finally Qwen3 Coder. This suggests that compiler error interpretation and debugging require broad reasoning capabilities rather than narrow programming knowledge. The ability to understand error messages, map them to code locations, and generate appropriate fixes appears to leverage the same general language understanding that makes these models effective across diverse tasks.

4.2.2 Small-scale model performance

The ICF results for small-scale models reveal a markedly different performance landscape compared to their large-scale counterparts. The small-scale models demonstrate severely limited capabilities in leveraging compiler feedback, with most achieving minimal or no improvement through the ICF process.

Gemma3-12b emerges as the only small-scale model capable of meaningful improvement through ICF, advancing from 1% base syntactic Pass@1 to 26% after 10 iterations—a substantial $26\times$ relative improvement, though still far below the performance of large-scale models. As visible in Fig. 5, Gemma3-12b demonstrates gradual, sustained improvement throughout the iteration process, suggesting some capacity for error interpretation and code correction within its limited parameter constraints. The remaining small-scale models exhibit severely constrained capabilities. Qwen2.5-Coder-14B-Instruct, despite having the largest parameter count among small models at 14B and being specifically designed for code generation, achieves minimal improvement from 1% to only 2%. This near-flat performance curve indicates fundamental limitations in understanding ArkTS syntax and compiler error messages, despite its coding specialization. DeepSeek-R1-Distill-Llama-8B shows marginal gains from 0% to 3%, while Mistral-7b Instruct demonstrates complete inability to benefit from ICF, maintaining 0% Pass@1 throughout all iterations. The confidence intervals for small-scale models reveal important reliability concerns. Gemma3-12b shows relatively wide confidence bands ($\pm 10 - 15\%$), indicating

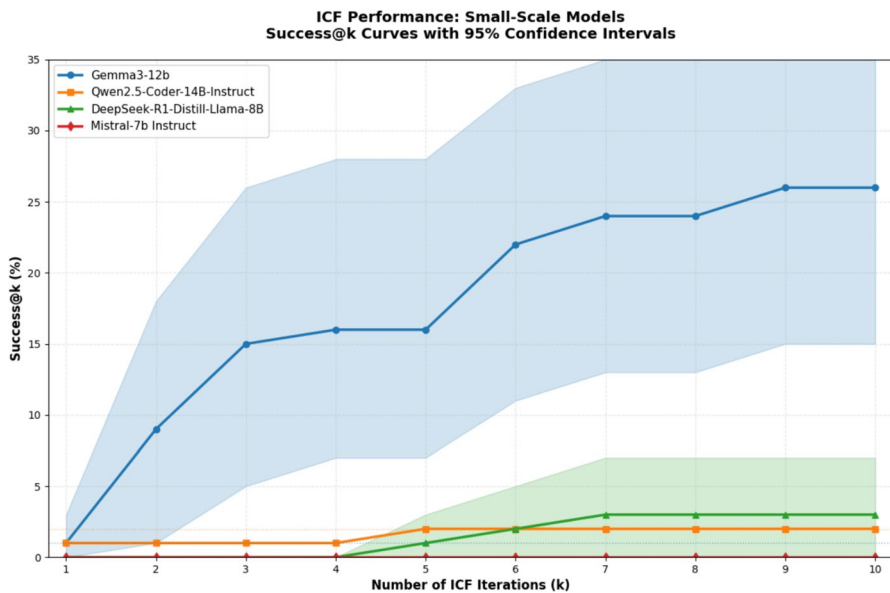


Fig. 5 ICF Success@k curves for small-scale models showing cumulative syntactic accuracy (compilation success) across iterations. The shaded regions represent 95% confidence intervals from bootstrap analysis with 2,000 iterations. Small-scale models demonstrate limited response to ICF, with only Gemma3-12b achieving meaningful improvement

inconsistent performance across different prompt variations. The other models maintain tight intervals around their low performance levels, suggesting consistent failure rather than variable success.

These results highlight fundamental limitations of resource-constrained models in domain-specific code generation tasks. The dramatic performance gap between small and large-scale models—with the best small model (Gemma3-12b at 26%) performing worse than the worst large model (Qwen3 Coder at 66%)—suggests that ArkTS code generation and compiler error interpretation require substantial reasoning capabilities that cannot be adequately compressed into smaller parameter spaces.

4.2.3 Error landscape and ICF effectiveness

Our analysis reveals that models encounter several categories of compilation errors during the ICF process, including API component hallucination (where models invent non-existent ArkTS components), property and type mismatches, fundamental language construct violations, and invalid module imports. The ICF methodology demonstrates varying effectiveness across these error types, with some categories showing high correction success rates while others remain challenging even with iterative feedback.

Importantly, the top-performing models—Claude Sonnet 4, Deepseek-V3-0324, and fine-tuned GPT-4o-mini (discussed in Section 3.7)—exhibit distinct error patterns and correction capabilities. A comprehensive analysis of these error categories,

their frequencies, and the ICF methodology's effectiveness in resolving them is presented in Section 4.4.

4.2.4 Key findings and implications

The plateau behavior observed across all models indicates that certain types of ArkTS compilation errors remain challenging even with iterative feedback. This creates a natural ceiling on ICF effectiveness for syntactic correctness, with final success rates ranging from 66% to 91% for large-scale models. The varying correction success rates across different error types reveal important insights about which programming challenges can be addressed through iterative feedback and which require deeper framework understanding. From a practical deployment perspective, these findings suggest that small-scale models are not viable candidates for ArkTS development assistance in their current form. The minimal success rates achieved even with ICF (26% maximum) fall well below the threshold of practical utility for developer tools. However, the modest success of Gemma3-12b indicates that targeted fine-tuning on ArkTS-specific data might yield better results for resource-efficient models, a direction we explore in Section 4.3. The contrast between large and small-scale model performance establishes a clear resource-performance trade-off in domain-specific code generation. While large models achieve practical utility through ICF, small models require alternative approaches such as specialized fine-tuning, retrieval-augmented generation, or hybrid architectures to become viable for ArkTS development assistance. These findings establish ICF as a highly effective methodology for achieving syntactic correctness in domain-specific code generation, while revealing important insights about the relationship between model architecture, specialization, and reasoning capabilities in novel programming domains.

4.3 Performance analysis of fine-tuned GPT-4o-mini model with different configurations

To evaluate the potential of targeted training for improving ArkTS code generation, we conducted a comprehensive fine-tuning experiment on GPT-4o-mini using OpenAI's supervised fine-tuning service. This analysis examines how different hyperparameter configurations affect the model's ability to learn the syntax and patterns of ArkTS, focusing specifically on batch size and epoch configurations.

4.3.1 Experimental setup and hyperparameter analysis

For our fine-tuning experiments, we systematically varied two key hyperparameters: batch size and the number of training epochs. The batch size parameter controls how many training examples are processed together in each update step, affecting both training stability and convergence behavior. We tested three batch size configurations: 8, 16, and 32. The epoch parameter determines how many complete passes through the training dataset the model performs, with more epochs potentially leading to better learning but also increased risk of overfitting to the specialized ArkTS training data. Our preliminary evaluations indicated that training beyond 4 epochs showed

no significant improvements and introduced potential overfitting risks. Therefore, we limited our experiments to 1, 2, and 4 epochs to balance learning effectiveness with computational efficiency. This systematic approach allowed us to identify the optimal configuration for adapting GPT-4o-mini to ArkTS code generation tasks.

4.3.2 Fine-tuning results and performance analysis

Table 4 presents the comprehensive results of our fine-tuning experiments across different batch size and epoch configurations. The results reveal several important patterns regarding the effectiveness of supervised fine-tuning for domain-specific language adaptation.

The most significant finding is the dramatic improvement achieved through fine-tuning. The best-performing configurations (GPT-4o-mini-ft-b32-e1 and GPT-4o-mini-ft-b32-e4) achieved 37% Pass@1, representing a remarkable 825% relative improvement over the baseline GPT-4o-mini performance of 4%. This substantial gain demonstrates the effectiveness of targeted training for domain-specific language adaptation. Analyzing the hyperparameter effects, we observe that larger batch sizes consistently outperform smaller ones. Models trained with batch size 32 achieve the highest performance, while those with batch sizes 8 and 16 show more variable and generally lower results. This pattern suggests that larger batch sizes provide more stable gradient estimates during training, leading to better convergence for this specialized domain. Regarding epoch count, the results reveal a nuanced relationship. For smaller batch sizes (8 and 16), increasing epochs often leads to performance degradation, indicating potential overfitting to the training data. However, for the optimal batch size of 32, both 1 and 4 epochs achieve the same peak performance of 37%, suggesting that this configuration reaches effective convergence quickly and maintains stability across extended training. Based on these results, we selected GPT-4o-mini-ft-b32-e1 as our optimal fine-tuned model configuration, as it achieves peak performance with the most efficient training duration. This configuration serves as the foundation for our subsequent analysis of fine-tuned model performance with ICF methodology.

Table 4 Performance analysis of GPT-4o-mini fine-tuning across different batch size and epoch configurations. The baseline GPT-4o-mini achieved 4% Pass@1 before fine-tuning

Model configuration	Batch size	Epochs	Pass@1
GPT-4o-mini-ft-b8-e1	8	1	0.35
GPT-4o-mini-ft-b8-e2	8	2	0.33
GPT-4o-mini-ft-b8-e4	8	4	0.33
GPT-4o-mini-ft-b16-e1	16	1	0.34
GPT-4o-mini-ft-b16-e2	16	2	0.31
GPT-4o-mini-ft-b16-e4	16	4	0.33
GPT-4o-mini-ft-b32-e1	32	1	0.37
GPT-4o-mini-ft-b32-e2	32	2	0.36
GPT-4o-mini-ft-b32-e4	32	4	0.37

Table 5 Performance progression of GPT-4o-mini across different configurations

Configuration	Pass@1	Improvement	Relative gain
Base GPT-4o-mini	4%	—	—
Fine-tuned GPT-4o-mini	37%	+33%	9.25×
Fine-tuned + ICF	82%	+78%	20.5×

4.3.3 Fine-tuned model performance and ICF enhancement

While the fine-tuned GPT-4o-mini model demonstrates improvement over its baseline performance, achieving 37% Pass@1 still reveals significant limitations when dealing with the full complexity of ArkTS development. During our post-analysis, we observed that the fine-tuned model generates more comprehensive and syntactically accurate code compared to the baseline version, demonstrating proper component structure and appropriate use of ArkTS decorators like `@Entry` and `@Component`.

To evaluate whether our ICF methodology can address the remaining challenges faced by the fine-tuned model, we applied the ICF approach to the optimally configured GPT-4o-mini-ft-b32-e1 model. Table 5 presents the comprehensive performance progression across three configurations.

Figure 6 presents the Success@k curve for the fine-tuned GPT-4o-mini model enhanced with ICF methodology. The results demonstrate substantial improvement, with fine-tuning alone producing a $9.25\times$ gain over baseline, transforming the model from nearly unusable (4%) to moderately functional (37%). The combination with ICF yields a $20.5\times$ improvement, reaching 82% Pass@1 at Success@10—a 122% relative improvement over the fine-tuned baseline.

The improvement pattern shows rapid initial gains, with Success@2 reaching 51% and Success@5 achieving 70%. The curve demonstrates convergence around

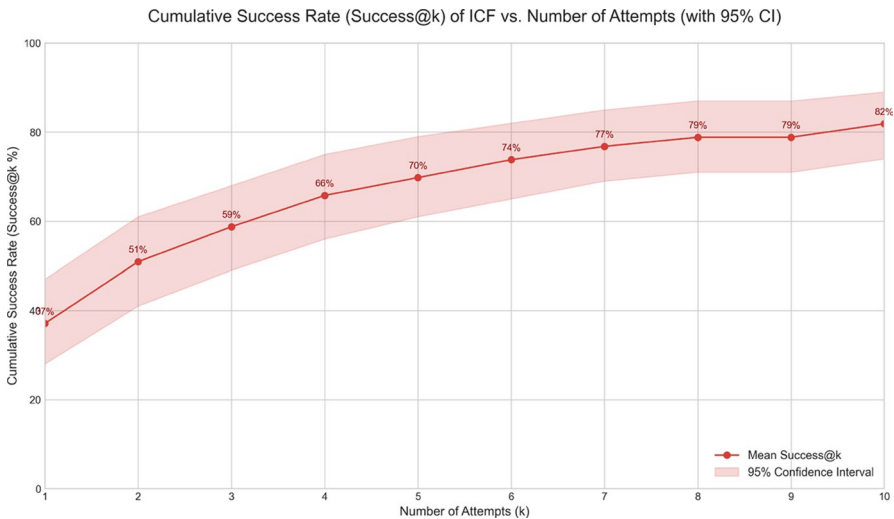


Fig. 6 Success@k curve for fine-tuned GPT-4o-mini with ICF methodology. The shaded region represents 95% confidence intervals from bootstrap analysis with 2,000 samples. The model demonstrates consistent improvement from 37% baseline to 82% at Success@10

attempts 8–9 at approximately 79%. This performance represents a critical milestone, as the model achieves high reliability that makes it practical for developer assistance applications. The combination of domain-specific fine-tuning with ICF proves particularly effective, addressing fundamental knowledge gaps through targeted training and remaining syntactic precision issues through autonomous error correction. This methodology successfully transforms a model with limited baseline capabilities into a practically viable tool, suggesting broad applicability for other low-resource programming languages and frameworks.

4.4 Syntactic error analysis

Based on our ICF evaluation results, we conducted a comprehensive error analysis of the three best-performing models: Claude Sonnet 4 (91% syntactic Pass@1), Deepseek-V3-0324 (83% syntactic Pass@1) and Fine-tuned GPT-4o-mini (82% syntactic Pass@1). This analysis examines the compilation errors these models encounter during the iterative correction process and their ability to achieve syntactic correctness through our ICF methodology. Throughout the ICF process, when a model's initial code attempt fails to compile, the ArkTS compiler provides error feedback, prompting the model to generate a corrected version. This iterative process continues until either the code compiles successfully or the maximum number of attempts is reached. We collected and analyzed all compilation error messages generated during these correction cycles across all instructions to understand the error landscape these models face during iterative debugging. Table 6 presents the syntactic error distribution based on comprehensive compiler output analysis across the entire ICF process. The percentages in Table 6 represent the relative frequency of each error category among all compilation errors encountered during ICF iterations. For example, when Claude Sonnet 4 shows 31% for API & Component Hallucination, this means that out of all compilation errors generated across all correction attempts for all instructions, 31% were classified as API & Component Hallucination errors. These percentages capture the complete error profile that models encounter while iteratively refining their solutions, including errors from early attempts that were subsequently fixed in later iterations, as well as persistent errors that remained unresolved.

Table 6 Syntactic error distribution across top models

Error category	GPT-4o-mini	Claude Sonnet 4	Deepseek-V3
API & Component Hallucination	47%	31%	54%
Property and Type Mismatch	28%	15%	14%
Fundamental Language Construct	10%	23%	3%
Invalid Module Imports & Scoping	7%	19%	25%
Naming Conflicts	4%	12%	4%
Component Hierarchy Errors	4%	—	—

4.4.1 Error category definitions

To systematically categorize failures, we implemented an automated parsing pipeline for the `hvisor` build logs. We defined regular expression matchers for standard ArkTS error codes (e.g., `ArkTS-Check-Error 1001`) and keywords. These raw logs were then mapped to the five high-level categories defined below. For example, errors containing “module not found” or “is not exported” were aggregated into “Invalid Module Imports,” while errors containing “type X is not assignable to type Y” were aggregated into “Property and Type Mismatch”.

API & Component Hallucination: The model invents non-existent ArkTS APIs, components, methods, or properties. This includes generating completely fabricated component names, methods that don’t exist in the ArkTS specification, or borrowing syntax patterns from other frameworks that are not available in ArkTS.

Property and Type Mismatch: The model uses valid, existing ArkTS components but applies incorrect parameter types, wrong property assignments, or incompatible type operations. These errors occur when components are used with syntactically incorrect type specifications.

Fundamental Language Construct: The model violates ArkTS-specific syntax rules that differ from standard TypeScript. This includes using prohibited any and unknown types, missing explicit type declarations for variables and function parameters, incorrect generic function usage, and violations of unique ArkTS architectural requirements such as `@CustomDialog` struct patterns and context handling in callbacks.

Invalid Module Imports & Scoping: The model attempts to import components from incorrect or non-existent module paths, or tries to import components that are already globally available in ArkTS and do not require import statements. This includes wrong import syntax and incorrect assumptions about module architecture.

Naming Conflicts: The model uses variable names that conflict with built-in component methods, reserved keywords, or existing method names within the ArkTS framework, leading to compilation ambiguity.

Component Hierarchy Errors: The model creates incorrect nesting, structural organization, or compositional patterns of UI components that violate ArkTS component architecture rules.

4.4.2 Fine-tuned GPT-4o-mini syntactic error patterns

The fine-tuned GPT-4o-mini model achieves 82% final syntactic Pass@1 through ICF. The model exhibits the highest rate of API & Component Hallucination errors (47%), frequently generating non-existent components such as:

ScrollController
GenericRefreshIndicator

for pull-to-refresh functionality, and attempting to use a `.filter()` method on Image components borrowed from web CSS patterns.

Property and Type Mismatch errors (28%) include persistent type incompatibility issues:

```
Type 'number' is not assignable to type  
'(value: ScaleOptions) => ...'
```

and incorrect state definitions like `@State height: string` that conflict with built-in `.height()` method expectations.

ICF Success Example: The model corrected radio button type errors in 2 attempts:

```
// Before (incorrect)  
Button('Radio 1').type('NormalButton')  
// After (correct)  
Button('Radio 1').type(ButtonType.Normal)
```

ICF Failure Example: Dialog functionality implementation failed across 10 attempts due to persistent misunderstanding of `@CustomDialog` architectural patterns.

4.4.3 Claude sonnet 4 syntactic error patterns

Claude Sonnet 4 achieves the highest syntactic Pass@1 (91%) through ICF. Our analysis identifies three distinct error patterns for this model:

- **Context Preservation (Fundamental Construct):** The model initially fails to capture the `this` context within callbacks, leading to “used before initialization” errors. It successfully resolves this in later iterations by switching to arrow function syntax.
- **Module Resolution (Imports):** It frequently hallucinates multi-file architectures (e.g., `import ... from '../model'`) for single-file prompts. The feedback loop effectively forces the model to inline these class definitions.
- **Layout Properties (Type Mismatch):** It confuses CSS flexbox properties with ArkTS equivalents (e.g., using `justifyContent` on a `Stack` component). These are corrected rapidly once the compiler flags the invalid property.

4.4.4 Deepseek-V3-0324 syntactic error patterns

Deepseek-V3 achieves 83% syntactic Pass@1 with the highest API & Component Hallucination rate (54%). The model shows particularly strong tendencies to invent module imports:

```
@ohos.pullToRefresh
```

and fabricate component properties:

```
.radioStyle()  
.selectedColor  
.activeColor
```

Invalid Module Imports & Scoping errors (25%) demonstrate persistent cycling through incorrect import paths:

```
@ohos.arkui.node  
@ohos.arkui.component
```

before recognizing that components are globally available.

The model shows the lowest rate of Fundamental Language Construct errors (3%), suggesting better grasp of basic ArkTS syntax compared to architectural understanding.

ICF Success Example: Image zoom functionality succeeded after resolving naming conflicts (scale to `imageScale`) and property corrections.

4.4.5 ICF effectiveness analysis

The ICF methodology demonstrates varying effectiveness across different error categories, revealing important insights about the nature of programming errors and the limits of iterative feedback correction. Property and Type Mismatch errors show the highest correction success rate at 85%, indicating that these errors are highly correctable to iterative feedback. When models receive explicit compiler messages about type incompatibilities, they can effectively identify the specific mismatched components and apply appropriate corrections. The concrete nature of type error messages provides clear guidance for models to adjust parameter types, property assignments, and type specifications. Invalid Module Imports and Scoping errors achieve a 78% correction success rate, demonstrating that import-related issues are generally resolvable through ICF. Models can effectively learn from compiler feedback about non-existent module paths and adjust their import strategies, often by recognizing that components are globally available or by correcting import syntax. The iterative process helps models understand the ArkTS module architecture through trial and error. In contrast, API & Component Hallucination errors achieve only a 62% correction success rate, representing the most challenging category for ICF methodology. This lower success rate indicates fundamental knowledge gaps that iterative feedback cannot resolve. When models invent non-existent APIs or components, compiler feedback can only indicate that these elements don't exist, but cannot guide the model toward the correct alternatives. The models lack the foundational knowledge of available ArkTS APIs to make appropriate substitutions. Fundamental Language Construct errors show mixed results, with higher correction rates for basic syntax issues but persistent failures for complex architectural patterns. Models can learn to avoid prohibited types like `any` and `unknown` through explicit compiler warnings, but struggle with deeper ArkTS-specific architectural requirements that require com-

prehensive framework understanding. These results highlight the diagnostic value of ICF methodology in identifying different types of model limitations. The varying correction success rates across error categories provide valuable insights into which programming challenges can be addressed through iterative feedback and which require different approaches. ICF proves particularly valuable for implementation-level corrections while revealing areas where models have fundamental knowledge gaps that require deeper understanding of the ArkTS framework.

4.5 Semantic validation results

Following our syntactic evaluation, we conducted a comprehensive semantic validation of the three best-performing models according to their Success@10 ICF results: Claude Sonnet 4 (91% syntactic Pass@1), Deepseek-V3-0324 (83% syntactic Pass@1), and fine-tuned GPT-4o-mini (82% syntactic Pass@1). As described in Section 3.4.2, this manual evaluation process assessed whether syntactically correct code implementations fulfill the complete functional requirements specified in the natural language instructions.

A designated human tester with expertise in ArkTS development manually evaluated each syntactically correct solution by deploying the generated applications to a HarmonyOS virtual device. The tester performed comprehensive functional reviews, comparing the application's actual behavior and appearance against every requirement detailed in the original prompt. An output was considered semantically correct only if it perfectly implemented all specified requirements, including UI layout, event handling, state changes, and interactive behaviors.

4.5.1 Semantic accuracy results

Table 7 presents the semantic validation results for our top-performing models. The semantic accuracy represents the percentage of syntactically correct solutions that also demonstrate complete functional correctness during runtime verification.

The results reveal that achieving syntactic correctness through ICF methodology translates to remarkably high semantic accuracy across all evaluated models. Claude Sonnet 4 demonstrates the highest semantic reliability at 96.7%, with only 3 semantic failures among its 91 syntactically correct outputs. Deepseek-V3-0324 achieves 91.6% semantic accuracy, while fine-tuned GPT-4o-mini reaches 90.2%. These high semantic accuracy rates indicate that the ICF methodology not only resolves compilation errors but also guides models toward functionally correct implementations.

Table 7 Semantic validation results. The semantic pass rate is calculated from the syntactically correct outputs

Model	Syntax pass	Sem. Err	Sem. Pass %
Claude Sonnet 4	91	3	96.7%
Deepseek-V3-0324	83	7	91.6%
Fine-tuned GPT-4o-mini	82	8	90.2%

4.5.2 Semantic failure pattern analysis

These semantic failure patterns reveal several key insights about the limitations of current LLM-based code generation for domain-specific frameworks. Models excel at generating basic UI structures and simple event handlers but struggle with multi-step interaction flows, such as complete drag-and-drop cycles or gesture-to-action bindings that require precise coordination between multiple system components. Additionally, while models successfully implement functional requirements, they often fail to meet precise visual specifications such as specific geometric shapes, complete styling transformations, or exact animation behaviors that require detailed understanding of framework rendering capabilities. Advanced state management scenarios involving complex data transformations, conditional rendering logic, or sophisticated user interaction patterns also frequently result in partial implementations. These findings demonstrate that while ICF methodology successfully bridges the gap to syntactic correctness and achieves high semantic accuracy, certain categories of requirements remain challenging for current LLM-based approaches in domain-specific development contexts.

5 Discussion

This section interprets the experimental findings, exploring the relationship between syntactic success and semantic accuracy in ArkTS code generation. We begin by examining specific behaviors observed during model evaluation, characterizing how current models handle visual reasoning and complex interaction logic. Subsequently, we outline the scope and methodological constraints of our study. The section concludes by proposing targeted research directions to address these observations and further advance automated development within the HarmonyOS ecosystem.

5.1 Results interpretation and failure analysis

The experimental findings reveal several broader insights into adapting LLMs for domain-specific programming languages. While high syntactic Pass@1 generally leads to high semantic accuracy, our manual validation revealed specific categories of tasks that remain challenging even for the best-performing models.

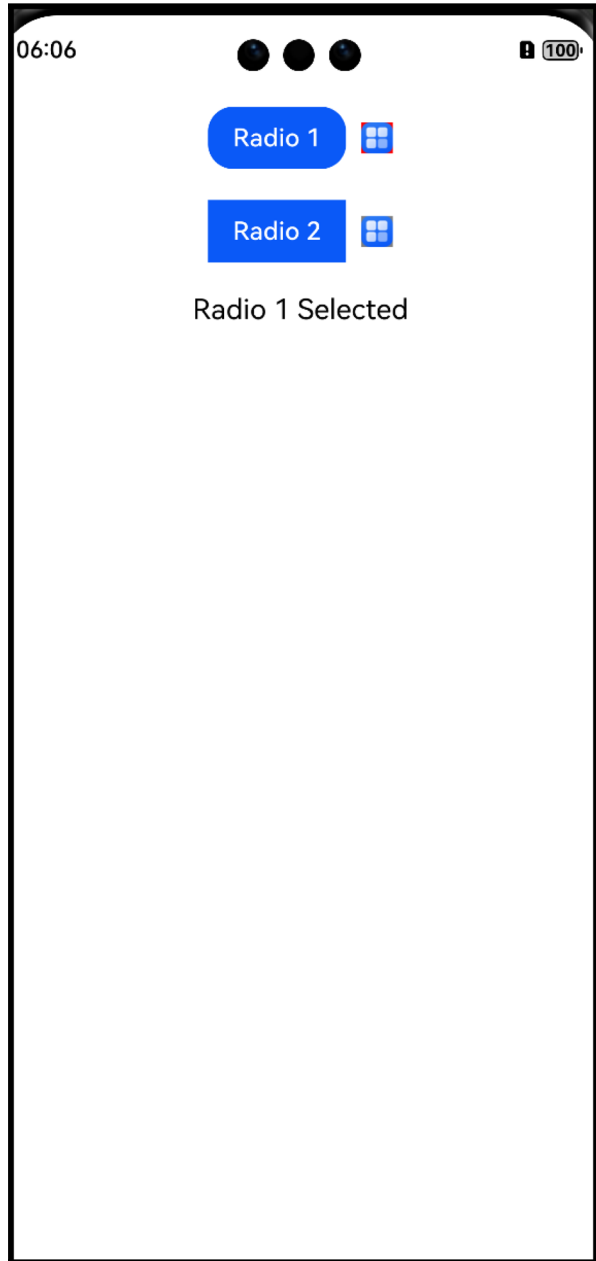
Spatial and visual reasoning limitations Models understand the syntax of UI components but often lack the “spatial reasoning” to render specific geometric shapes using code properties. For example, when Claude Sonnet 4 was tasked with creating a “Triangle Indicator,” it implemented a functional swipe mechanism but failed to render the visual triangle, resorting to a standard circular indicator (see Fig. 7). Similarly, fine-tuned models often failed to apply complete visual styling, such as changing only the border color of a radio button instead of the entire component (Fig. 8).

Complex interaction logic While simple event handlers are generated correctly, multi-step interaction flows often break. For instance, in a “Drag and Drop” task, the

Fig. 7 Triangle Indicator Failure: The model used a default circular indicator instead of the requested triangle shape



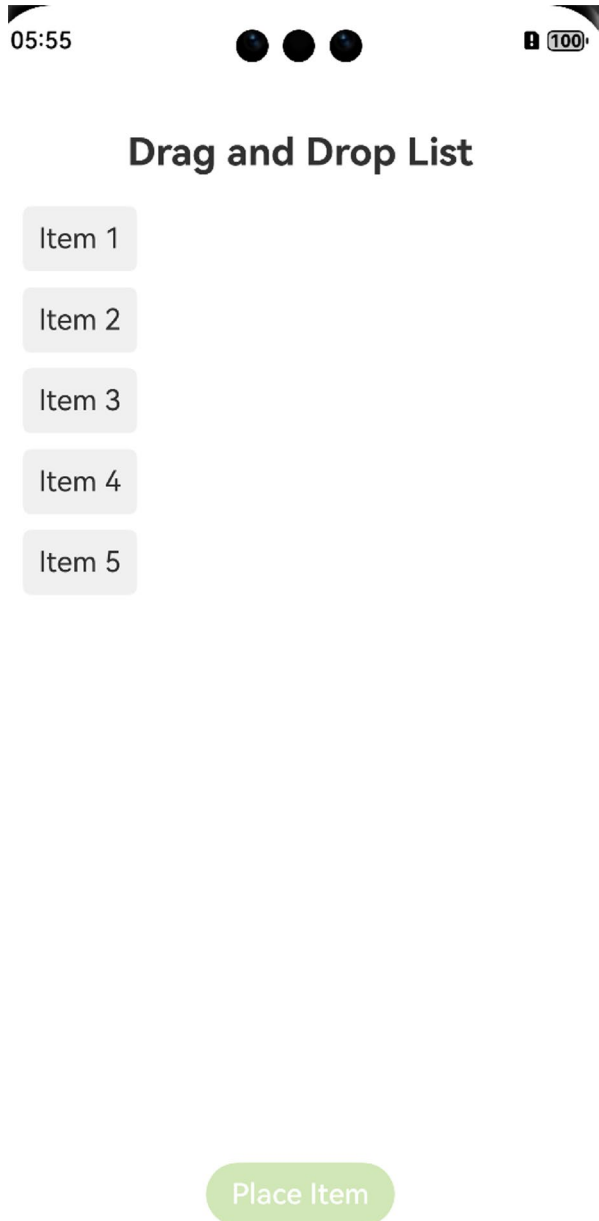
Fig. 8 Styling Failure: Radio button changed border color only, missing the full visual specification



model correctly implemented the drag initiation but failed to handle the drop event data transfer (Fig. 9). This suggests that while ICF fixes syntax, it cannot easily correct logic errors that do not trigger compiler warnings.

Other observed failures included the misinterpretation of state management behavior (e.g., changing text content instead of removing it) and incomplete implementation

Fig. 9 Drag and Drop Interface - Partial functionality where drag works but drop fails



of gesture-to-action bindings (e.g., pull-to-refresh UI appearing but not triggering the refresh action). These findings suggest that future approaches may need richer feedback sources beyond compiler errors, such as automated UI testing or execution-trace analysis.

5.2 Limitations

Runtime performance metrics Our evaluation focuses on syntactic validity (Pass@1) and functional correctness. We did not measure runtime metrics such as Frame Rate (FPS), memory consumption, or battery impact. Since ArkTS relies on an Ahead-of-Time (AOT) compiler that optimizes standard UI components, we assume syntactically valid native code performs within acceptable bounds, but future work should verify this assumption using on-device profiling tools. While our test cases use only universal ArkUI components that are compatible across all HarmonyOS devices, we did not evaluate responsive layout adaptation across different screen sizes (e.g., smartwatch vs. tablet) or performance on resource-constrained hardware.

Manual validation bottleneck The semantic analysis relied on manual human testing, where experts deployed applications to virtual devices and verified functional correctness against requirements. This approach was necessary because ArkTS UI validation requires verifying visual appearance and interaction behaviors that cannot be captured through standard automated unit tests. However, this manual process limits scalability for large-scale evaluation. Future work could explore Vision-Language Models (VLMs) to automate visual verification by comparing rendered screenshots against natural language specifications which is discussed in Section 5.3.

Model scope While we evaluated a diverse range of models, the rapid release cycle of LLMs means newer models (e.g., Llama-3, GPT-4o) were not included. However, the patterns of failure—specifically API hallucination due to the domain gap—are likely to persist due to the general lack of ArkTS data in pre-training corpora.

5.3 Future work

Several research directions emerge from our updated findings:

Automated test case generation Reducing reliance on manual semantic validation is critical. Future work should explore using Vision-Language Models (VLMs) to act as automated testers, capturing screenshots of the rendered ArkTS UI and comparing them against the natural language requirements to verify visual correctness (e.g., detecting if a triangle is actually rendered).

Knowledge graph-enhanced generation (RAG) Building structured ArkTS API and component graphs to support retrieval-augmented generation (RAG) could mitigate the API hallucinations observed in Section 4.4. Selective retrieval strategies would help manage context size limitations, especially for small-scale models.

Hybrid small-model solutions Pairing small adapted models with ArkTS-specific RAG pipelines offers a pathway to near-large-model performance at reduced cost. However, limited context size in small models constrains how much retrieved information can be incorporated at once.

6 Conclusion

We presented the first systematic evaluation of LLMs for ArkTS code generation in the HarmonyOS ecosystem, introducing the *ArkTS-Test* and the Iterative Compilation Feedback (ICF) methodology. Our updated experiments confirm that base models achieve limited syntactic Pass@1, with the best large-scale model reaching only 35% on first attempts. The application of ICF raised performance for large-scale models to between 66% and 91% syntactic Pass@1, with Claude Sonnet 4 achieving the highest reliability. In our experiments, the fine-tuned GPT-4o-mini combined with ICF achieved syntactic accuracy levels comparable to those of some large models. From a practical perspective, large-scale models with ICF currently provide the highest reliability for ArkTS development assistance, but smaller models may be preferable in scenarios where cost efficiency is prioritized. The persistence of API hallucination across all configurations underscores the importance of integrating structured ArkTS knowledge to improve correctness. Addressing automated semantic validation via VLMs and enabling scalable, multi-module project synthesis remain important directions for advancing fully AI-assisted development in specialized software ecosystems. To facilitate reproducibility and further research, we have made the *ArkTS-Test*, the fine-tuning training dataset, and our implementation code for the ICF methodology publicly available on GitHub at <https://github.com/cemaytekin/llm-finetuning-arkts/tree/main>.

A Appendix

A.1 Reproducibility

To facilitate reproducibility and enable further research in this domain, we have made our constructed training and test datasets, along with all related experimental materials, publicly available on GitHub at <https://github.com/cemaytekin/llm-finetuning-arkts/tree/main>. This repository contains the complete instruction-code pairs, evaluation scripts, and documentation necessary to replicate our experimental methodology and extend our work to additional model configurations or framework versions.

Acknowledgements We are grateful for the highly collaborative environment at the Huawei Research Center in Turkey. This work would not have been possible without the constant encouragement and synergy within our Mobile Service Development Center (MSDC) AI team across the Istanbul and Ankara offices. We deeply appreciate all the contributions that have led to the completion of this paper.

Author Contributions Primary author, Mehmet Cem Aytekin is responsible from all parts of the manuscript.

Data Availability <https://github.com/cemaytekin/llm-finetuning-arkts/tree/main>.

Declarations

Competing interests The authors declare no competing interests.

References

- Ahmad, W.U., Ficek, A., Samadi, M., Huang, J., Noroozi, V., Majumdar, S., Ginsburg, B.: Opencodestruct: A large-scale instruction tuning dataset for code llms (2025). [arXiv:2504.04030](https://arxiv.org/abs/2504.04030)
- Anthropic.: Introducing claude 3.7 sonnet (2025a). <https://www.anthropic.com/news/claude-3-7-sonnet>
- Anthropic.: A new era of reasoning: Claude sonnet 4 (2025b). <https://www.anthropic.com/news/claude-sonnet-4>
- Arakelyan, S., Das, R.J., Mao, Y., Ren, X.: Exploring distributional shifts in large language models for code analysis. In: Conference on Empirical Methods in Natural Language Processing (2023). <https://doi.org/10.48550/arXiv.2303.09128>
- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al.: Program synthesis with large language models (2021). [arXiv:2108.07732](https://arxiv.org/abs/2108.07732)
- Chen, B., Zhang, F., Nguyen, A., Zan, D., Lin, Z., Lou, J.G., Chen, W.: Codet: Code generation with generated tests (2022). [arXiv:2207.10397](https://arxiv.org/abs/2207.10397)
- Chen, H., Zhang, Y., Li, X., Wang, Y., Sun, Y.: Designcoder: Hierarchy-aware ui code generation via llms. In: Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems, ACM (2025)
- DeepSeek AI.: Deepseek-v3-chat technical report (2025). <https://deepseek.com/technologies/v3-chat>
- DeepSeekAI.: Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning (2025). <https://huggingface.co/deepseek-ai/DeepSeek-R1>
- Google.: Gemma 3: The next generation of open models (2025). <https://ai.google/discover/gemma3/>
- Google DeepMind.: Gemini 2.5 pro: Unlocking the next wave of ai-powered development (2025). <https://deepmind.google/technologies/gemini/2.5-pro/>
- Gui, L., Chen, B., Liu, Z., Li, M., Wang, Q.: Uicopilot: Hierarchical planning for complex user interface code generation. In: Proceedings of the 2025 ACM Symposium on User Interface Software and Technology, ACM (2025)
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, D., Song, D., Steinhardt, J.: Measuring coding challenge competence with apps (2021). [arXiv:2105.09938](https://arxiv.org/abs/2105.09938)
- Hu, X., Kuang, K., Sun, J., Yang, H., Wu, F.: Leveraging print debugging to improve code generation in large language models (2024). <https://doi.org/10.48550/arXiv.2401.05319> [arXiv:2401.05319](https://arxiv.org/abs/2401.05319)
- Huawei Developers.: TypeScript to ArkTS Cookbook: Migration Guide (2024). <https://developer.huawei.com/consumer/en/doc/harmonyos-guides/typescript-to-arkts-migration>. Accessed 01 Dec 2025
- Husain, H., Wu, H.H., Gazit, T., Allamanis, M., Brockschmidt, M.: Codesearchnet challenge: Evaluating the state of semantic code search (2019). [arXiv:1909.09436](https://arxiv.org/abs/1909.09436)
- Jiang, A.Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D.S., de las Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., et al.: Mistral 7b (2023a). [arXiv:2310.06825](https://arxiv.org/abs/2310.06825)
- Jiang, J., Wang, F., Shen, J., Kim, S., Kim, S.: A survey on large language models for code generation (2024). [arXiv:2406.00515](https://arxiv.org/abs/2406.00515)
- Jiang, X., Dong, Y., Wang, L., Shang, Q., Li, G.: Self-planning code generation with large language models. ACM Trans. Softw. Eng. Method. (2023). <https://doi.org/10.1145/3672456>
- Kulal, S., Saad, F., Singh, R., Chidambaram, V., Kohli, P.: Spoc: Search-based pseudocode to code. In: Advances in Neural Information Processing Systems, pp 11906–11917 (2019)
- Li, K.S., Yuan, Y.: Large language models as test case generators: Performance evaluation and enhancement (2024). <https://doi.org/10.48550/arXiv.2404.13340> [arXiv:2404.13340](https://arxiv.org/abs/2404.13340)
- Liang, Z., Zhou, X., Qian, K., Li, H., Zhang, D.: Waffle: Structure-aware code generation from webpage screenshots. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics (2024)
- Lu, S., Guo, D., Ren, S., Wang, J., Tang, D., Duan, N., Zhou, M.: Codexglue: A benchmark dataset and open challenge for code intelligence (2021). [arXiv:2102.04664](https://arxiv.org/abs/2102.04664)
- Nashid, N., Sintaha, M., Mesbah, A.: Retrieval-based prompt selection for code-related few-shot learning. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), IEEE, pp 2450–2462 (2023)
- OpenAI.: Introducing gpt-4o mini: Efficiency and power at scale (2025). <https://openai.com/blog/gpt-4o-mini/>
- OpenHarmony.: Openharmony application development docs - ui section (2024). <https://gitee.com/openharmony/docs/tree/1561150137b851ddb08bef83b02872bac9ecc780/en/application-dev/ui>
- Qwen Team.: Announcing qwen 2.5: A new family of open-source language models (2025a). <https://qwen.ai/blog/qwen2-5/>

- Qwen Team.: Qwen3-coder: Advancing agentic coding with a new moe language model (2025b). <https://qwen.ai/blog/qwen3-coder/>
- Wu, T., Zhao, L., Li, C., Xu, W.: Layoutcoder: Layout tree guided user interface code generation. In: Proceedings of the 2025 Web Conference, ACM (2025)
- Yeo, S., Ma, Y., Kim, S.C., Jun, H., Kim, T.: Framework for evaluating code generation ability of large language models. ETRI J. (2024). <https://doi.org/10.4218/etrij.2023-0357>
- Yin, P., Neubig, G.: Mining contextualized translation examples for model translation. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, pp. 1387–1392 (2018)
- Zhang, S., Chen, Z., Shen, Y., Ding, M., Tenenbaum, J., Gan, C.: Planning with large language models for code generation. In: International Conference on Learning Representations (2023). <https://doi.org/10.48550/arXiv.2303.05510>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Mehmet Cem Aytekin¹ · Fatma Gizem Yılmaz¹ · Mustafa Umut Demirezen²

✉ Mehmet Cem Aytekin
mehmet.cem.aytekin@huawei.com

Fatma Gizem Yılmaz
fatma.gizem.calli@huawei.com

Mustafa Umut Demirezen
umut.demirezen@huawei.com; umut@demirezen.tech

¹ MSDC Department DC, Huawei Türkiye R&D Center, İstanbul, Türkiye

² MSDC Department DC, Huawei Türkiye R&D Center, Ankara, Türkiye